BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 3

Roll no: 51 PRN No: 12411033 Name of Student: Manjiri Kulkarni

Subject Name : Deep Learning

Problem Statement :

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

1. Start.
2. Import required libraries.
3. Load and preprocess the dataset.
4. Create a neural network model.
5. Initialize learning rates and number of epochs.
6. Perform forward propagation to generate predictions.
7. Calculate loss/error.
8. Apply backpropagation to update weights.
9. Train the model for selected epochs.
10. Evaluate model accuracy and loss.
11. Compare performance for different learning rates and epochs
12. Stop.

Code :

```
import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

from tensorflow.keras.optimizers import Adam
```

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

#  Load Dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
class_names = [
"T-shirt/top",
   "Trouser",
   "Pullover",
   "Dress",
   "Coat",
   "Sandal",
   "Shirt",
   "Sneaker",
   "Bag",
   "Ankle boot"
]

print("Training Data Shape :", x_train.shape)
print("Testing Data Shape  :", x_test.shape)

# Display Sample Images
```

```
plt.figure(figsize=(8,6))

for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(class_names[y_train[i]])
    plt.axis("off")

plt.suptitle("Fashion MNIST Sample Images")
plt.show()
#  Data Preprocessing
x_train = x_train.reshape(-1,784)/255.0
x_test = x_test.reshape(-1,784)/255.0

print("\nAfter Reshaping")
print("Training :",x_train.shape)
print("Testing  :",x_test.shape)

# Create Model
def create_model(lr):

    model = Sequential([
        Dense(128,activation='relu',input_shape=(784,)),
        Dense(64,activation='relu'),
        Dense(10,activation='softmax')
    ])
```

```
    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model


# Learning Rate Analysis
learning_rates=[0.1,0.01,0.001]

lr_results=[]

for lr in learning_rates:

    model=create_model(lr)

    model.fit(
        x_train,
        y_train,
        epochs=5,
        batch_size=128,
        verbose=0
    )
```

```
    loss,accuracy=model.evaluate(
        x_test,
        y_test,
        verbose=0
    )

    lr_results.append([lr,accuracy])

lr_df=pd.DataFrame(
    lr_results,
    columns=["Learning Rate","Accuracy"]
)

print("\nLearning Rate Comparison")
print(lr_df)

plt.figure(figsize=(6,4))

plt.plot(
    lr_df["Learning Rate"],
    lr_df["Accuracy"],
    marker='o',
    linewidth=2
)
```

```
plt.xscale("log")
plt.xlabel("Learning Rate")
plt.ylabel("Accuracy")
plt.title("Learning Rate vs Accuracy")
plt.grid(True)

for i in range(len(lr_df)):
    plt.text(
        lr_df["Learning Rate"][i],
        lr_df["Accuracy"][i],
        f"{lr_df['Accuracy'][i]:.4f}"
    )

plt.show()

# Epoch Analysis
epochs_list=[5,10,20]

epoch_results=[]

for ep in epochs_list:

    model=create_model(0.001)

    model.fit(
        x_train,
```

```
        y_train,
        validation_split=0.2,
        epochs=ep,
        batch_size=128,
        verbose=0
    )

    loss,accuracy=model.evaluate(
        x_test,
        y_test,
        verbose=0
    )

    epoch_results.append([ep,accuracy])

epoch_df=pd.DataFrame(
    epoch_results,
    columns=["Epochs","Accuracy"]
)

print("\nEpoch Comparison")
print(epoch_df)

plt.figure(figsize=(6,4))

plt.plot(
```

```
    epoch_df["Epochs"],
    epoch_df["Accuracy"],
    marker='o',
    linewidth=2
)

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Epochs vs Accuracy")
plt.grid(True)

for i in range(len(epoch_df)):
    plt.text(
        epoch_df["Epochs"][i],
        epoch_df["Accuracy"][i],
        f"{epoch_df['Accuracy'][i]:.4f}"
    )

plt.show()

#  Train Final Model
model=create_model(0.001)

history=model.fit(
    x_train,
    y_train,
```

```
    validation_split=0.2,
    epochs=20,
    batch_size=128
)



#  Accuracy Curve
plt.figure(figsize=(7,5))

plt.plot(history.history['accuracy'],label="Training Accuracy")
plt.plot(history.history['val_accuracy'],label="Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")

plt.legend()
plt.grid(True)
plt.show()

#  Loss Curve
plt.figure(figsize=(7,5))

plt.plot(history.history['loss'],label="Training Loss")
plt.plot(history.history['val_loss'],label="Validation Loss")
```

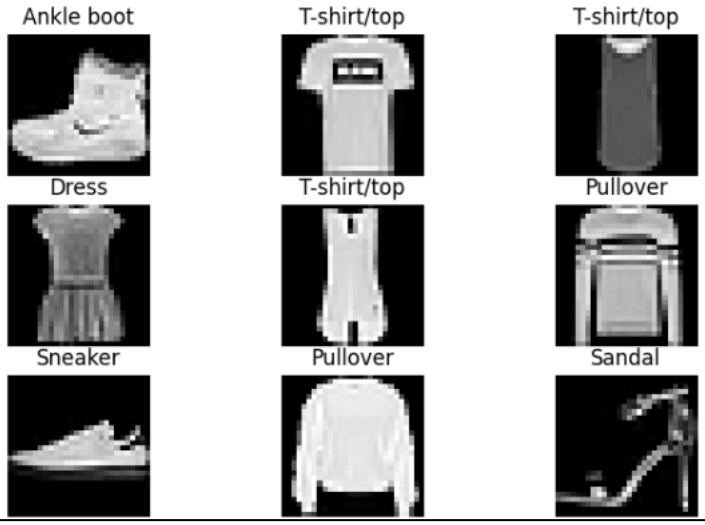
```
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")

plt.legend()
plt.grid(True)
plt.show()

 # Model Evaluation
loss,accuracy=model.evaluate(
   x_test,
   y_test,
   verbose=0
)

print("\nFinal Test Accuracy :",accuracy)
print("Final Test Loss     :",loss)

#  Prediction Visualization
prediction=model.predict(x_test)

plt.figure(figsize=(12,6))

for i in range(10):

   plt.subplot(2,5,i+1)
```

```
    plt.imshow(
        x_test[i].reshape(28,28),
        cmap="gray"
    )

    pred=np.argmax(prediction[i])

    color="green" if pred==y_test[i] else "red"

    plt.title(
        f"P:{class_names[pred]}\nA:{class_names[y_test[i]]}",
        fontsize=8,
        color=color
    )

    plt.axis("off")

plt.suptitle("Sample Predictions")
plt.show()

#Confusion Matrix

predicted_labels=np.argmax(
    prediction,
    axis=1
```

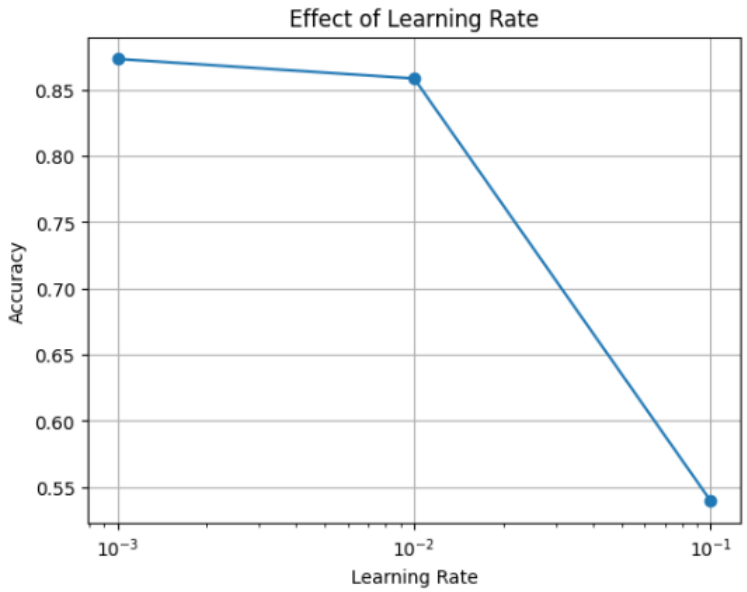
```
)

cm=confusion_matrix(
   y_test,
   predicted_labels
)

plt.figure(figsize=(10,8))

disp=ConfusionMatrixDisplay(
   confusion_matrix=cm,
   display_labels=class_names
)

disp.plot(xticks_rotation=45)
plt.title("Confusion Matrix")
plt.show()
```

Output :

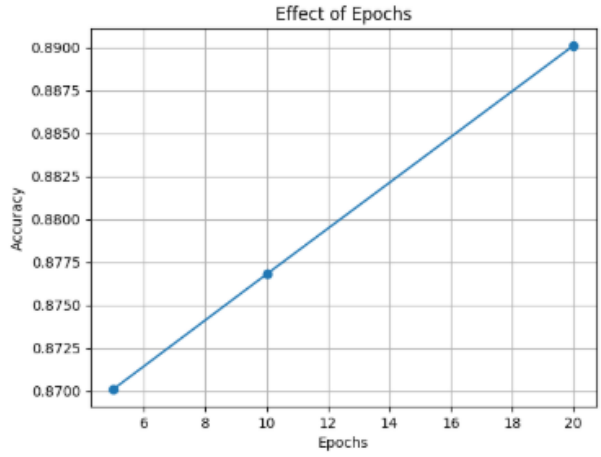
```
Training Data Shape: (60000, 28, 28)
Testing Data Shape: (10000, 28, 28)
```



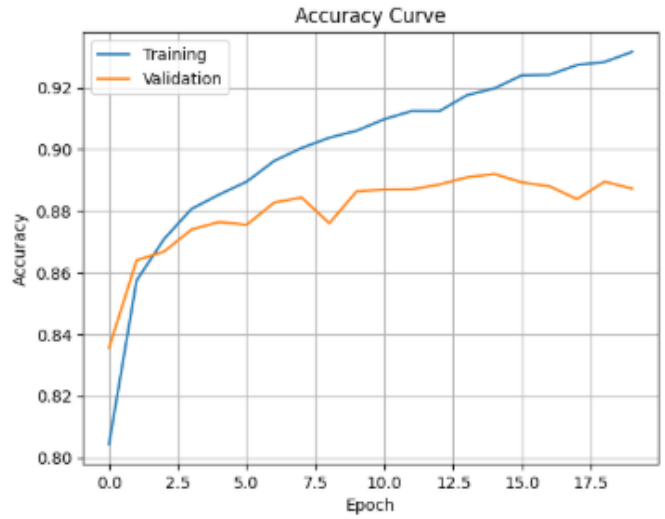

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_d
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
   Learning Rate  Accuracy
0          0.100    0.5396
1          0.010    0.8582
2          0.001    0.8730
```

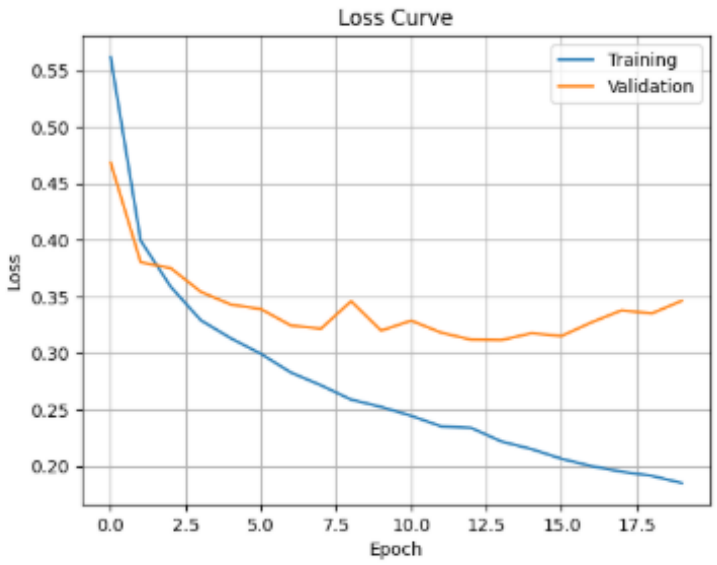


```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential mo
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
   Epochs  Accuracy
0       5    0.8701
1      10    0.8768
2      20    0.8901
```




```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential mo
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.8044 - loss: 0.5619 - val_accuracy: 0.8356 - val_loss: 0.4686
Epoch 2/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.8574 - loss: 0.3996 - val_accuracy: 0.8640 - val_loss: 0.3804
Epoch 3/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.8709 - loss: 0.3587 - val_accuracy: 0.8668 - val_loss: 0.3752
Epoch 4/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.8807 - loss: 0.3291 - val_accuracy: 0.8740 - val_loss: 0.3543
Epoch 5/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8853 - loss: 0.3132 - val_accuracy: 0.8764 - val_loss: 0.3430
Epoch 6/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8895 - loss: 0.2994 - val_accuracy: 0.8755 - val_loss: 0.3390
Epoch 7/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.8962 - loss: 0.2828 - val_accuracy: 0.8827 - val_loss: 0.3244
Epoch 8/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9004 - loss: 0.2716 - val_accuracy: 0.8843 - val_loss: 0.3217
Epoch 9/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9038 - loss: 0.2589 - val_accuracy: 0.8760 - val_loss: 0.3459
Epoch 10/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9060 - loss: 0.2525 - val_accuracy: 0.8864 - val_loss: 0.3199
Epoch 11/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9098 - loss: 0.2446 - val_accuracy: 0.8869 - val_loss: 0.3288
Epoch 12/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9316 - loss: 0.1851 - val_accuracy: 0.8873 - val_loss: 0.3463
```






```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8808 - loss: 0.3716
Test Accuracy: 0.8808000087738037
```

Loss Curve